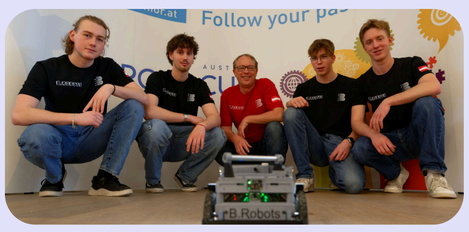
RESCUE MAZE JUNIOR
B.ROBOTS



Achievements

2024 - World Cup
- Participated

2025 - Austrian Open
- 1st place
- Best Documentation
- Best Poster

2025 - World Cup
- 5th place
- Community Award

2026 - Austrian Open
- 1st place
- Best Documentation
- Best Poster

ABOUT US

We are **B.Robots Seniors**, an Austrian engineering team from HTL Bulme in Graz, Styria.

Paul Charusa - Hardware, CAD, PCB
“I am the team captain and was responsible for the CAD design and parts of the PCB layout.”

Thomas Rauch - Camera, Video
“I trained and programmed the Machine Vision Model for optimal detection of victims.”

Vincent Rohkamm - Mapping, UI, Software Integration
“I developed the mapping system, the UI and the main structure of the programs”

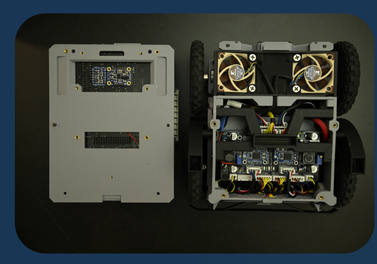
Florian Wiesner - Sensor Integration, Driving
“I integrated the sensors and developed the driving algorithms and parts of the main program.”

HARDWARE OVERVIEW

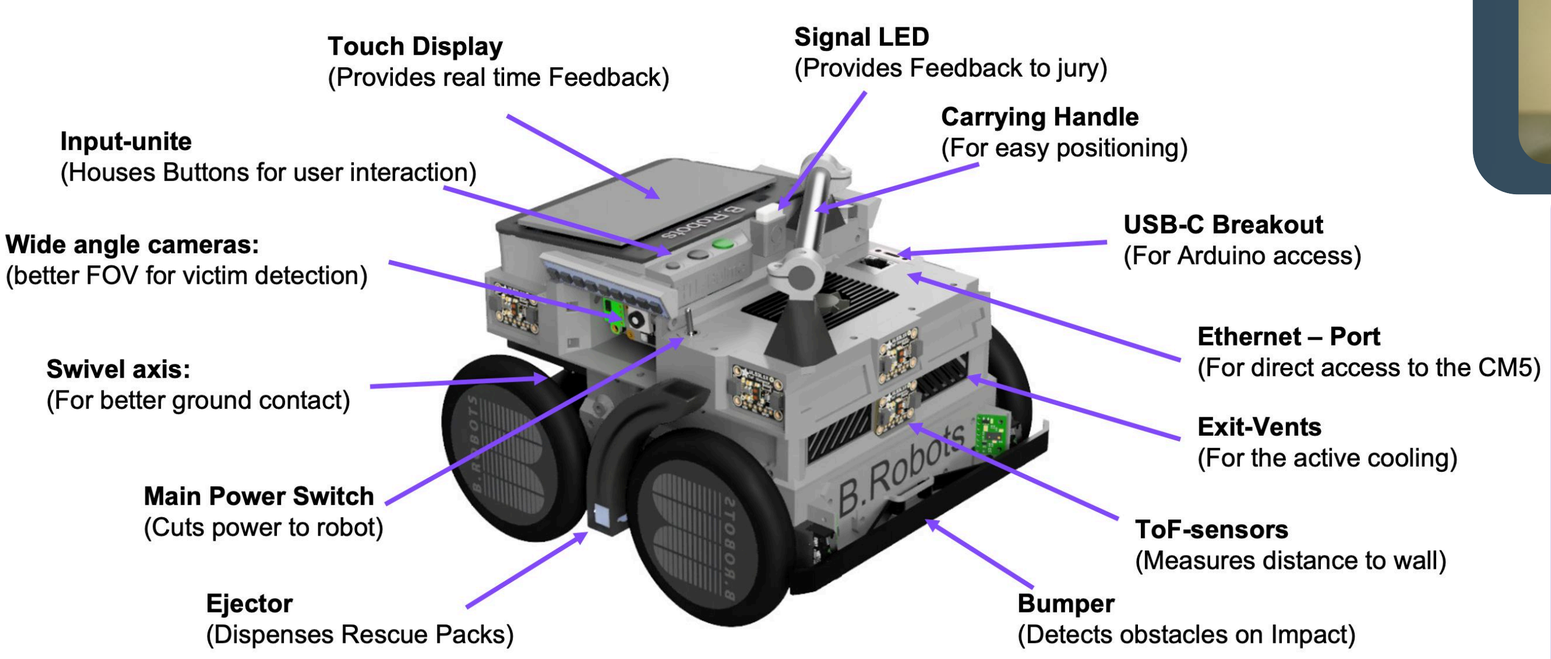
- **Main Controller**
  Arduino Giga R1
- **Drive Train**
  4 DC motors with encoders
- **Rescue Kit Ejection**
  2 servo motors
- **Wall Detection and Distance Measurement**
  10 ToF sensors
- **Victim Detection**
  RasPi CM5 with TensorFlow-Light
- **Sensor Connections**
  Custom PCBs
- **Lighting for Camera support**
  18 Neopixel RGBW LEDs to improve visibility
- **Main Display**
  4” TFT-LCD display with interactive UI
- **Inclination Measurement**
  Adafruit BNO055 IMU
- **Frontal Collision Detection (Bumper)**
  Two mechanical switches for collision detection

Innovative Design Solutions

Our Robot consists of 3 modular layers, each with its own PCB. The layers connect via pin headers and can be easily seperated by removing just a few screws, simplifying repairs and maintenace.



Supported by two ball bearings, the swivel axis rotates freely and self-aligns to ground obstacles, ensuring stable positioning and reliable sensor readings by improving ground contact.



Cameras

Victim detection is managed by a Raspberry Pi 5 Compute Module with two wide-angle cameras that utilize a custom TensorFlow Lite model. To optimize power consumption and processing overhead, the Compute Module is connected to Time-of-Flight (ToF) sensors positioned next to the cameras. These sensors only trigger the cameras when a wall is detected. Once a victim is successfully detected and validated, the Raspberry Pi transmits the data via UART to the Arduino Giga for signaling and mapping reference.

SOFTWARE OVERVIEW

Technologies & programming language

The software of the Robot is coded in C++ and Python, using VS-Code, PlatformIO and Github.

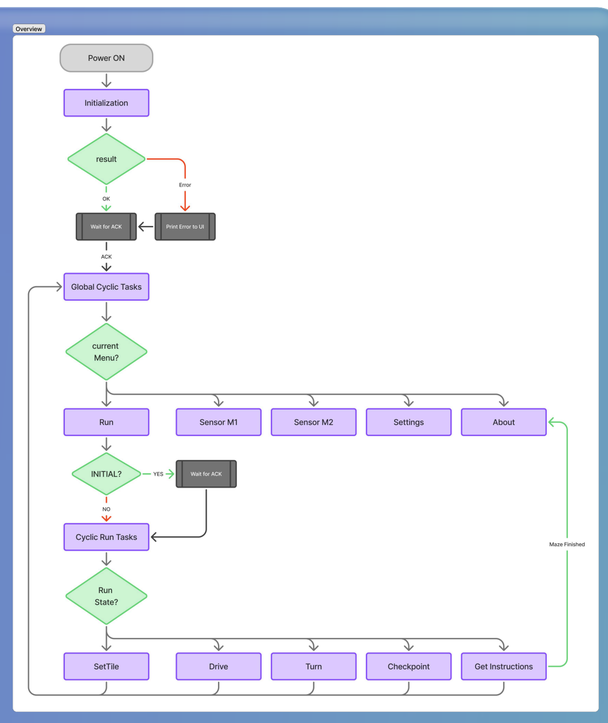
General Software Architecture

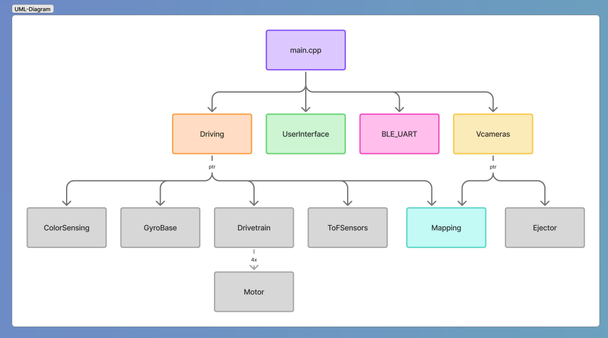
The architecture is built on two nested state machines. The outer RobotState machine handles the operating mode, while the inner RunState machine executes the competition run as a deterministic cycle. Every loop iteration first executes the global cyclic tasks, and during a run, additional cyclic run tasks are layered on top. This guarantees that sensor data is always fresh.
Each subsystem is realized as its own C++ class inside a dedicated library. Dependencies are wired explicitly by pointer injection during initialization. This removes hidden global singletons, so every interaction path is visible in the UML on the right, and each module can be tested in isolation.

Innovations

Non-blocking sensor framework:
Modified ToF library with continuous ranging and data-ready polling instead of blocking reads.

Continous Traversal:
The robot does not stop on each tile when driving straight. In the middle of each tile, information about walls, the floor, and ramps is saved and sent to the mapping system. The robot only stops if a wall is in front of it or if the mapping commands a turn.





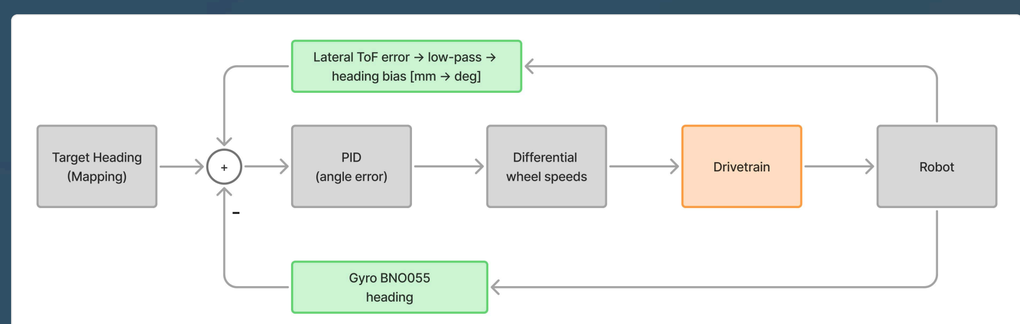
INTERESTING ALGORITHMS

A* pathfinding

Navigation uses an A* search over a 3D tile graph for multiple maze levels. Tiles have a traversal cost. The planner avoids expensive tiles automatically whenever a cheaper route exists. Exploring and returning home use one consistent mechanism. The cost model also drives obstacle handling. When a bumper is triggered, the current tile's cost is increased or the route is locked. The active path is invalidated. The A* plans around the obstruction on the next cycle.

```
Instructions Mapping::GetInstruction(){
    if path is finished OR bumper was triggered:
        //Finish run
        if return_home is active AND currentPosition == start:
            return MazeFinished

        // reset Bumper Flag
        bumper was triggered = false

        // find next target position with BFS
        target = findNextTarget()

        // calculate cheapest path to target with A*
        path = AStar(currentPosition, target)

        if target is unreachable:
            return unreachable

        // convert path to instructions
        convert path to instructions and save in path
        store instructions in path
        set pathIndex = 1
        return first instruction from path

    else:
        return next instruction from path
}
```

PID controller - drive & turn



Conditional-integration ant-windup dt measured derivative with kick protection gains tuned offline (Ziegler-Nichols)